Reviewer Pack — Minimal Rank of Quadratic Residues in Number Theory vs Resol…

Entry 769cbb7e7999 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 19:45:30 UTC

Minimal Rank of Quadratic Residues in Number Theory vs Resolution Proof Length for Tseitin Formulas

Entry ID: `769cbb7e7999` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 19:45:30 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Quadratic Residues) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

> For any instance of a Tseitin formula with n variables, the length of the shortest Resolution proof is upper bounded by O(n^2 log n / log log n), where the logarithm base is the number of quadratic residues modulo n.

Rationale (proposer's reasoning):

> Quadratic residues provide a way to measure the algebraic structure in a field. If this structure can be exploited to simplify the resolution process, it could lead to new insights into the complexity of Tseitin formulas. Additionally, the connection between number theory and logic is well-established, with potential for cross-disciplinary applications.

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `4cf62a641d97a621`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the average shortest Resolution proof length across 30 seeds is within ±10% of O(n^2 log n / log log n), calculated with the number of quadratic residues modulo n as the logarithm base.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.80	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def is_quadratic_residue(a, n):
    return pow(a, (n - 1) // 2, n) == 1

def count_quadratic_residues(n):
    count = 0
    for a in range(1, n):
        if is_quadratic_residue(a, n):
            count += 1
    return count

def generate_tseitin_formula(n):
    variables = [f'x{i}' for i in range(n)]
    clauses = []
    for i in range(n):
        clauses.append(f'{variables[i]}')
        clauses.append(f'-{variables[i]}')
    for i in range(1, n):
        clauses.append(f'{variables[i-1]} {variables[i]}')
        clauses.append(f'-{variables[i-1]} -{variables[i]}')
    return ' '.join(clauses)

def resolution_length(formula):
    clauses = formula.split()
    literals = set()
    while True:
        new_literals = set()
        for clause in clauses:
            if not clause:
                continue
            literal, *rest = clause.split()
            if literal.startswith('-'):
                literal = literal[1:]
                if literal in literals:
```

```
                    literals.remove(literal)
                else:
                    new_literals.add(literal)
            else:
                if literal in literals:
                    return len(clauses) + 1
                else:
                    literals.add(literal)
        if not new_literals:
            break
        clauses.extend([f'-{l}' for l in new_literals])
    return len(clauses)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_length = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):  # Ensure at least 5 instances per size
            formula = generate_tseitin_formula(n)
            length = resolution_length(formula)
            total_length += length
            instances_tested += 1

    average_length = Fraction(total_length, instances_tested)

    expected_length = n_values[-1]**2 * math.log(n_values[-1], count_quadratic_residues(n_values[-

    conjecture_holds = abs(average_length - expected_length) <= 0.1 * expected_length
    counterexample = "" if conjecture_holds else f"n={n_values[-1]}, avg_length={average_length},

    return {
        "metric_name": "Resolution Proof Length",
        "metric_value": float(average_length),
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_length = sum(r["metric_value"] for r in results) / len(results)
    std_length = math.sqrt(sum((r["metric_value"] - mean_length)**2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
```

```
        print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fr
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fr
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n_values[-1], avg_length, expected_length\" fir
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_330101dc.py", line 98, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_330101dc.py", line 79, in run_trial
    expected_length = n_values[-1]**2 * math.log(n_values[-1], count_quadratic_residues(n_values[-
1])) / math.log(math.log(n_values[-1], count_quadratic_residues(n_values[-1])))
                                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed due to a ZeroDivisionError before producing data, which prevents us from evaluating the conjecture's support condition. | next: Investigate and fix the ZeroDivisionError in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13997
2	propose	ollama_remote	glm4:latest	0	0	12799
3	preregistration	ollama_remote	glm4:latest	0	0	9914
4	novelty	ollama_remote	glm4:latest	0	0	8873
5	novelty	ollama_remote	glm4:latest	0	0	8472
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16241
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9988
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10369
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11144

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	judge	ollama_remote	glm4:latest	0	0	11436

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 113231 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/769cbb7e7999.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/769cbb7e7999.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/769cbb7e7999.tar.gz` (if generated)